July 2023 CSE 102
Pointers and Multidimensional Arrays

Full Marks: 10 Duration: 25 minutes

Submission Guidelines

Create a C file named according to the following convention: **2205xyz.c**. Upload this C file for your submission.

Problem Description

Your task is to implement basic operations on two-dimensional matrices using pointers, with a focus on performing matrix multiplication. Direct use of array indexing notation (i.e., **array[i][j]**) is prohibited; instead, work exclusively with pointers. You may use standard library functions only for input/output purposes.

Task 1: User Input Function [2 Marks]

- Implement a user function that takes two matrices as input.
- Take inputs for these matrices. Avoid using array indexing notation or temporary variables for input; you must use pointers.

Task 2: Matrix Multiplication Function [4 Marks]

- Implement a user function that takes two matrices as input and dynamically declares a new matrix to store the result of multiplying the two matrices. The resultant matrix will have dimensions **r1*c2**.
- Implement the multiplication operation without using array indexing notation or temporary variables; you must use pointers.
- Return the resultant matrix using pointers.

Task 3: Print Resultant Matrix [2 Marks]

- Implement a user function that takes a matrix as input and prints the resultant matrix after multiplication. Ensure that you use pointers to access and print elements in the correct order.

Task 4: Main Function [2 Marks]

In the main function:

- Take two sets of integers **r1**, **c1** and **r2**, **c2** as input, representing the number of rows and columns for two matrices, respectively. [For matrix multiplication, **c1** must equal **r2**.]

- Dynamically allocate memory for two matrices of sizes **r1*c1** and **r2*c2** using pointers.
- Call input function.
- Call matrix multiplication function.
- Call print function for printing the resultant matrix.

Constraints

- You are assured that inputs will be valid and conform to the rules of matrix multiplication (**c1** equals **r2**).
- Avoid using the **[]** operator. Manipulate pointers directly.
- Ensure proper memory management to prevent memory leaks.
- You cannot use global variables.
- Input elements and matrix dimensions are integers within practical limits for this exercise. Assume **r1**, **c1**, **r2**, and **c2** will not exceed **100**.

Input	Output
2 2 2 2 1 2 3 4 5 6 7 8	19 22 43 50